

Framework Review — ICITY Foods Automation

This document reviews a **typical junior Selenium POM project** (common patterns seen at ~1.5 years experience) and explains how this refactored framework addresses each issue.

> **Note:** No existing Selenium project was found in the workspace. This framework was built against the ICITY Foods React app and reflects the refactor target described in your requirements.

1. Overall project structure

What is wrong (typical junior setup)

...

src/test/java/

```
LoginTest.java    // driver setup inside test
HomeTest.java     // duplicate waits and locators
Utils.java        // one large utility class with everything
config.properties // mixed with test code
```

...

- Tests, page objects, driver logic, and reporting are mixed together.
- No clear package boundaries (`base`, `pages`, `utilities`, etc.).
- Hard to scale when new modules or browsers are added.

Why it should be changed

Professional teams separate **framework code** (main) from **test code** (test). This makes reuse, code reviews, and CI maintenance easier.

Improved implementation

...

src/main/java/com/icityfoods/automation/

base/ → lifecycle + shared page behavior

config/ → externalized settings

pages/ → UI abstraction only

utilities/ → reusable Selenium helpers

listeners/ → TestNG hooks

reports/ → Extent report setup

src/test/java/.../tests/ → assertions + business flows only

src/test/resources/ → config, testng.xml, log4j2.xml

...

Best practices

- Keep page objects free of assertions.
- Keep tests thin — call page methods, assert outcomes.
- Store environment data in `config.properties`, not hard-coded strings.

2. DriverFactory

What is wrong (typical)

```
```java
```

```
public class LoginTest {
 WebDriver driver;
```

```

@BeforeMethod
public void setup() {
 System.setProperty("webdriver.chrome.driver", "C:/drivers/chromedriver.exe");
 driver = new ChromeDriver();
 driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
 Thread.sleep(3000);
}
}
'''

```

Issue	Impact
Hard-coded driver path	Breaks on other machines/CI
Driver created in every test class	Duplicate setup/teardown
`Thread.sleep()`	Flaky, slow tests
Implicit wait only	Unreliable for dynamic React UI
No ThreadLocal	Parallel execution breaks

### ### Why it should be changed

Driver management is **infrastructure**, not test logic. One factory ensures consistent browser config and safe parallel runs.

### ### Improved implementation — `utilities/DriverFactory.java`

- Uses **WebDriverManager** — no manual driver binaries.
- **ThreadLocal<WebDriver>** — parallel-ready.
- **Zero implicit wait** — forces explicit waits in `ElementHelper`.
- Supports `chrome`, `firefox`, `edge` via config.
- Headless mode controlled by `config.properties`.

### ### Best practices

- One driver per test method/thread.
- Always `quit()` in `@AfterMethod`.
- Never share WebDriver across threads without ThreadLocal.

---

## ## 3. BaseTest

### ### What is wrong (typical)

```
```java
@AfterMethod
public void tearDown() {
    driver.quit(); // no screenshot on failure
}
```
```

- No logging of test start/end.
- No failure screenshot.
- Each test class repeats the same boilerplate.

### ### Improved implementation — `base/BaseTest.java`

```
```java
@BeforeMethod → init driver + open baseUrl + log test name
@AfterMethod → capture screenshot on FAILURE + quit driver + log status
```
```

### ### Best practices

- Use `alwaysRun = true` on teardown so browser closes even when test fails.
- Attach screenshot path to `ITestResult` for the listener/report.

---

## ## 4. ElementHelper (utility layer)

### ### What is wrong (typical)

```
```java
// Duplicated in every page class
driver.findElement(By.id("email")).click();
Thread.sleep(2000);
driver.findElement(By.id("email")).sendKeys("test@test.com");
```
```

- Duplicate interaction code across pages.
- No reusable wait strategy.
- No logging when actions fail.
- Alert/window/dropdown handling copy-pasted.

### ### Improved implementation — `utilities/ElementHelper.java`

| Method                             | Purpose                                          |
|------------------------------------|--------------------------------------------------|
|                                    |                                                  |
| `click()`                          | Waits for clickable, handles stale element retry |
| `jsClick()`                        | JS fallback for obscured elements                |
| `sendKeys()`                       | Clear + type with visibility wait                |
| `getText()`                        | Trimmed text with explicit wait                  |
| `scrollToElement()`                | Scroll into view before action                   |
| `selectDropdownByVisibleText()`    | Standard ` <select>` support</select>            |
| `acceptAlert()` / `dismissAlert()` | Alert handling                                   |
| `switchToWindowByTitle()`          | Multi-window support                             |

All waits use:

```
```java
new WebDriverWait(driver, Duration.ofSeconds(timeout))
    .until(ExpectedConditions.visibilityOfElementLocated(locator));
```
```

### Best practices

- **Never use `Thread.sleep()`** in framework or tests.
- Wrap Selenium exceptions at the utility layer and log context (locator, action).
- Keep timeout values in config, not magic numbers in every method.

---

## 5. BasePage

### What is wrong (typical)

```
```java
public class HomePage {
    WebDriver driver;

    public HomePage(WebDriver d) { this.driver = d; }

    public void clickMenu() {
        driver.findElement(By.xpath("/html/body/div[1]/div/nav/a[3]")).click();
    }
}
```
```

- Absolute XPath — breaks with any DOM change.
- Navigation logic duplicated in HomePage, MenuPage, ContactPage.
- No shared wait helper.

### Improved implementation — `base/BasePage.java`

- Holds `WebDriver` + `ElementHelper`.
- Provides **\*\*dynamic nav locator\*\***:

```
```java
By.xpath("//header//nav//a[normalize-space()='%s']")
```
```

- `PageFactory.initElements()` ready for `@FindBy` if needed later.

### Best practices

- Prefer semantic locators: `name`, `aria-label`, stable CSS classes.
- Use parameterized XPath for repeated patterns (nav links, table rows).
- Return next page object from navigation methods (optional fluent flow).

---

## 6. Page objects (HomePage, MenuPage, ContactPage, OurStoryPage)

### What is wrong (typical)

| Anti-pattern | Example |

|-----|-----|

| Assertions in page class | `assertTrue(driver.getTitle().equals("Home"))` |

| Hard-coded test data in page | `sendKeys("john@test.com")` |

| Locators as inline strings in methods | Different XPath in every method |

| Mixed UI + API logic | Page class calling REST APIs |

### Improved implementation

Each page class:

1. Declares **\*\*private static final By\*\*** locators at the top.
2. Exposes **\*\*intent-based methods\*\*** (`searchFood`, `submitForm`, `selectCountry`).
3. Returns data via getters — **\*\*no TestNG assertions inside pages\*\***.

Example — `ContactPage.java`:

```
```java
private static final By NAME_INPUT = By.cssSelector("input[name='name']");
private static final By COUNTRY_TRIGGER = By.cssSelector("button[aria-label='Select country']");

public void selectCountry(String countryName) {
    helper.click(COUNTRY_TRIGGER);
    helper.click(By.xpath(String.format(
        "//div[@role='listbox']/button[.//span[normalize-space()='s']]", countryName)));
}
```
```

### Locator strategy used

| Element          | Locator type                        | Why                              |
|------------------|-------------------------------------|----------------------------------|
| -----            | -----                               | -----                            |
| Form fields      | `input[name='email']`               | Stable HTML attribute            |
| Nav links        | XPath with link text                | Readable, matches UI labels      |
| Country dropdown | `aria-label` + role-based XPath     | Works with custom React dropdown |
| Menu search      | `#menu-search input[type='search']` | Scoped to section ID             |

### Best practices

- One page class  $\approx$  one application page/screen.
- Method names describe **\*\*user intent\*\***, not Selenium commands.
- Avoid indexing (`/div[3]/div[1]`) unless no alternative exists.

---



## ## 7. ConfigReader

### ### What is wrong (typical)

```
```java
```

```
String url = "http://localhost:5173"; // copied in 5 test classes
```

```
FileReader fr = new FileReader("C:/config/config.properties"); // absolute path
```

```
```
```

- Hard-coded URLs and credentials in tests.
- Config loaded repeatedly in every class.
- No validation when keys are missing.

### ### Improved implementation — `config/ConfigReader.java`

- Loads `config.properties` **\*\*once\*\*** from classpath.
- Typed helpers: `get()`, `getInt()`, `getBoolean()`.
- Throws clear error if required key is missing.

### ### Best practices

- Never commit real credentials; use `.env` or CI secrets for sensitive data.
- Keep test data (names, emails) in config for easy environment switching.

---

## ## 8. ScreenshotUtils

### ### What is wrong (typical)

- Screenshots taken manually in some tests, skipped in others.
- Saved to random folders with no naming convention.
- Not linked to Extent Report.

### ### Improved implementation

- Triggered automatically from `BaseTest` on failure.
- Timestamped file names: `testName\_yyyyMMdd\_HHmms.png`
- Path stored on `ITestResult` for listener attachment.

### ### Best practices

- Capture screenshot **\*\*before\*\*** quitting the browser.
- Sanitize test names for valid file names.

---

## ## 9. TestListener + ExtentManager

### ### What is wrong (typical)

```
``java
```

```
// Extent created inside every test method
```

```
ExtentReports extent = new ExtentReports();
```

```
extent.attachReporter(new ExtentHtmlReporter("report.html"));
```

```
``
```

- Multiple report instances overwrite each other.
- No pass/fail/skip status mapping.
- Screenshots not embedded in report.

### ### Improved implementation

- **\*\*ExtentManager\*\*** — singleton report instance.
- **\*\*TestListener\*\*** — implements `ITestListener`:
  - `onTestStart` → create Extent test
  - `onTestSuccess/Fail/Skip` → log status
  - `onTestFailure` → attach screenshot from `ITestResult`
  - `onFinish` → flush report

Registered in `testng.xml`:

```
``xml
```

```
<listener class-name="com.icityfoods.automation.listeners.TestListener"/>
```

...

### ### Best practices

- Use ThreadLocal for ExtentTest in parallel runs.
- Flush report once at suite end, not per test.

---

## ## 10. Test classes

### ### What is wrong (typical)

```
```java
@Test
public void test1() {
    driver.get("http://localhost:5173");
    Thread.sleep(5000);
    driver.findElement(By.xpath("...")).click();
    Assert.assertEquals(driver.getTitle(), "Home");
}
```
```

- Full flow + locators + setup inside test method.
- No test description.
- Assertions without meaningful messages.

### ### Improved implementation

```
```java
@Test(description = "Verify top navigation links route to expected pages")
public void verifyTopNavigationLinks() {
    HomePage homePage = new HomePage(DriverFactory.getDriver());
    Assert.assertTrue(homePage.isHeroVisible(), "Home page did not load");
}
```

```

    homePage.clickNavLink("Menu");

    MenuPage menuPage = new MenuPage(DriverFactory.getDriver());

    Assert.assertTrue(menuPage.getPageHeadingText().contains("Where Great Food"), "Menu page heading mismatch");
}
...

```

Test coverage included

Test class	Scope
`HomePageTest`	Hero section smoke
`NavigationTest`	All navbar routes
`MenuPageTest`	Search + veg filter
`ContactFormValidationTest`	Client-side validation only (no live email send)

Best practices

- One logical assertion group per test (or closely related checks).
- Use `@Test(description = "...")` for report readability.
- Always pass a **failure message** to Assert methods.

11. pom.xml

What is wrong (typical)

- Missing dependency versions (reproducibility issues).
- No surefire plugin → TestNG suite not picked up.
- Random JARs added without version properties.

Improved implementation

- Centralized versions in ``<properties>``.
- Dependencies: Selenium, TestNG, WebDriverManager, Log4j2, ExtentReports, Commons IO.
- `maven-surefire-plugin` points to `testng.xml`.

Best practices

- Pin versions in pom — don't rely on transitive latest.
- Run `mvn clean test` in CI the same way as locally.

12. Logging (Log4j2)

What is wrong (typical)

```
```java
System.out.println("Clicked login button");
```
```

- No log levels, no timestamps, no file output.
- Cannot filter debug vs error in CI logs.

Improved implementation

- `log4j2.xml` → console + file appenders.
- Logger used in `DriverFactory`, `ElementHelper`, `BaseTest`, `ScreenshotUtils`.

Best practices

- `logger.info()` for test lifecycle events.
- `logger.debug()` for element interactions.
- `logger.error()` with exception stack trace on failures.

Summary — junior vs professional framework

| Area | Junior pattern | This framework |
|-----|-----|-----|
| Driver setup | Per test class, manual path | DriverFactory + WebDriverManager |
| Waits | Thread.sleep / implicit only | WebDriverWait + ExpectedConditions |
| Locators | Absolute XPath | CSS, name, aria-label, dynamic XPath |
| Duplication | Copy-paste in every class | BasePage + ElementHelper |
| Config | Hard-coded values | config.properties + ConfigReader |
| Reporting | Console only | Log4j2 + Extent + screenshots |
| Structure | Flat test folder | base / pages / utilities / listeners / reports |
| Scalability | Low | Add page + test class per feature |

How to run

```bash

*# Terminal 1 — app under test*

cd icityfoods/icity-foods && npm run dev

*# Terminal 2 — automation*

cd icityfoods-automation && mvn clean test

```

Reports: `test-output/reports/ExtentReport.html`